

Hermes: Towards Coinference-Centric Serving for LLM Applications

Anonymous Author(s)

Abstract

Applications based on Large Language Models (LLMs) have become increasingly popular in recent years. To accomplish a high-level mission, LLM applications usually issue a series of correlated LLM inferences and non-LLM tasks, involving dynamic resource volumes, hybrid resource backends, as well as diverse performance objectives. Existing LLM systems are oblivious to such characteristics, being suboptimal when serving LLM applications. In this paper, we abstract the resource demands of an LLM application into a *coinference*, and further design *Hermes*, a coinference-centric LLM serving system. *Hermes* models a coinference as a list of functional stages—with built-in properties depicting inter-stage dependencies and intra-stage demands. *Hermes* further applies the Gittins policy to optimize inter-coinference scheduling performance against diverse objectives. Moreover, for fast serving of individual LLM applications, *Hermes* leverages the coinference-level demand model to coordinately provision the desired resources in various aspects like KV cache and non-LLM tools. We have implemented *Hermes* atop vLLM, and experiments with diverse LLM applications confirm that it can effectively improve the application serving efficiency, reducing the average completion time by up to 90%.

1 Introduction

Large Language Models (LLMs) [21, 33, 70] have been widely adopted in various domains [24, 31, 66]. Given the prohibitively high cost to train newer-generation LLMs, it is now more reachable to boost the power of LLMs by *scaling-out* (i.e., building LLM-based applications) instead of by *scaling-up* (i.e., adopting more advanced models) [15]. That is, by combining a set of LLM inference requests and also integrating with the non-LLM tools, users can improve the LLM performance in aspects like input length [45, 50], output reliability [26, 44] and functional flexibility [61, 62].

An LLM application refers to a group of LLM inferences, as well as the accompanying non-LLM tasks, that collaborate to accomplish a high-level mission for end users (§2.1). Given the prevalence of such LLM applications, it becomes critical to serve them efficiently in clusters. In particular, compared with traditional workloads, LLM applications have three distinct characteristics. First, the resource demand of LLM applications—like the text generation length and the total inference number—are dependent on the runtime inputs and

uncertain a priori. Second, serving LLM applications often requires coordinated provisioning of diverse resource backends (to support both LLM and non-LLM computations) [5, 48]. Third, LLM applications may have heterogeneous service objectives—some focus on end-to-end mission completion time [6, 45], yet some others care about instantaneous token generation speed [52].

When serving LLM applications, existing works fail to work well in both inter-application scheduling and intra-application execution. Most works in the literature focus on accelerating individual LLM inferences without awareness of the inter-request correlations [22, 30, 34, 46, 53], suffering suboptimal end-to-end performance due to inter-request delay. While some recent works can improve the performance of LLM applications [35, 51, 52, 68], they focus on specific application types (e.g., chatbot) or optimization aspects (e.g., KV cache reuse), without dealing with the common characteristics of LLM applications. Consequently, due to demand dynamicity, advanced scheduling algorithms like Shortest-Remaining-Processing-Time (SRPT) cannot be adopted when scheduling multiple LLM applications, nor can it be feasible to support diverse objectives. Meanwhile, without coordinated resource provisioning, an LLM application may be impeded on any resource backend due to contention or improper configuration.

In this paper, we seek to enhance the scheduling and execution efficiency of LLM applications, and the key is to leverage request correlations. To be specific, we propose an abstract called *coinference*, which organizes the correlated inferences of an LLM application in proper structure, and also records the non-LLM resource demands between LLM inferences. We note that with the built-in [3] or customized [13] notations in LLM requests, it is easy for the LLM backend to identify the correlated inferences belonging to the same application. The coinference abstraction can help to directly optimize the application-level scheduling objectives. Moreover, for a given LLM application, our empirical study shows that its resource demand is relatively stable across different trial runs with diverse inputs; therefore, with coinference-level demand modeling, we can more precisely predict the demand information of the entire application as well as of each request, which can further facilitate application scheduling and resource provisioning.

Based on the coinference abstraction, we design *Hermes*, a coinference-centric serving system for LLM applications, with the objective of achieving improved efficiency in both inter-application scheduling and intra-application execution.

In our Hermes design, we address the challenging characteristics of LLM applications (i.e., demand dynamicity, objective diversity, and resource hybridity) with three modules: coinference demand modeling, inter-coinference scheduling, and intra-coinference resource provisioning.

First, we devise a general and precise method for coinference demand modeling. For generality, we model a coinference as a list of *stages*, where each stage represents a functional unit with one or multiple parallel requests. In each stage, we record a series of properties to express request dependencies, looping status and even non-LLM demands. For preciseness, we choose to record the resource demands in each aspect (e.g., input/output token length, request parallelism) as a distribution rather than a single constant, trying to preserve the panoramic demand information; we also notice the demand correlations between dependent requests and further adopt correlation analysis to enhance the demand prediction accuracy at runtime.

Second, we design a scheduling algorithm that can minimize the average completion time of LLM applications, and further extend it to support diverse objectives. Confronting uncertain demands with distribution-like description, the classical SRPT algorithm is no longer optimal. For scheduling LLM applications, Hermes instead applies Gittins policy [18, 59], which has been proven optimal for jobs with unknown demand but known demand distributions. Moreover, when concurrently serving applications with SLO objectives (e.g., on deadline or instantaneous generation speed), Hermes keeps identifying and prioritizing the ones with a high risk of SLO violation, and the remaining applications are scheduled under the default SRPT-like policy.

Third, for the efficient execution of individual applications, we propose coinference-aware clairvoyant resource provisioning, applying the demand information from coinference modeling for management of the diverse resource backends. When managing the storage space of KV cache [47] and LoRA adaptors [41], we propose clairvoyant cache management to determine the cache eviction order based on the coinference priority and the prefetching moments based on the profiled distribution of inter-stage gaps. When managing non-LLM backends like CPU (to launch docker container) and network (to conduct RAG), we propose clairvoyant queueing that propagates the coinference priority to each resource backend. In cases with multiple LLM backends with heterogeneous batch size configurations, we propose clairvoyant backend selection, which means adaptively selecting the backend for each stage based on the profiled request parallelism. These methods can ensure that an LLM application can have fast completion without being impeded on any resource backends.

We have implemented Hermes with over 4,100 lines of Python code atop vLLM, a popular LLM serving framework.

We evaluate Hermes performance against mainstream schedulers with a diverse set of modern LLM applications. Experimental results show that Hermes can improve the average application completion time by up to 90.1% while providing a near-optimal SLO guarantee. Ablation studies also confirm the effectiveness of our approaches in tackling demand dynamicity as well as in efficient resource provisioning on diverse backends.

2 Background and Motivation

2.1 LLM Applications: A Primer

2.1.1 The trend towards LLM applications. Large language models (LLMs) [21, 33, 70] stand out as a general technique whose intelligence capability can potentially be applied to various domains [31, 38]. Yet, even though AI models are evolving rapidly [57, 65], it is commonly recognized that a single LLM request is often deficient when addressing realistic problems. For example, the input context windows of typical LLMs are often of limited sizes (less than 10M tokens) [25, 50], thus processing a large document would require issuing multiple parallel inference requests. Meanwhile, LLMs are plagued by the problem of *hallucination* [43], meaning that the output of a single LLM request may be unfaithful or nonsensical, and some additional LLM requests are then required to ensure LLM output quality (for example, with FacTool [26] verifications or with self-reflection [44]). Besides, the built-in knowledge and capability of LLMs are usually insufficient in professional occasions, thus various non-LLM components (e.g., knowledge retrieval [48], third-party tools [62] or other small models [61]) are commonly adopted to augment LLM capabilities [19], as integrated in LLM agents like AutoGPT [32] and MetaGPT [40]. To summarize, to enlarge the input length, improve the output reliability, and integrate more functionalities, LLM practitioners often need to conduct multiple inferences with the necessary non-LLM computations, which form a compound workflow.

We call the above compound workflow—with multiple LLM requests and even non-LLM tasks to accomplish a realistic mission—as an *LLM application*. LLM applications are rapidly emerging in recent years [38, 51, 68], and have become an increasingly prevalent workload type in production clusters. Fig. 1 presents the structures and descriptions of ten typical LLM applications¹ studied in this paper.

¹We deem multi-turn conversation as a special application whose mission is to help users obtain the desired information via iterative LLM interaction. Moreover, in solution design, we do not particularly differentiate AI agents [32, 40] from other applications, because the definition of agent itself is vague, and the typical components of AI agents, like ReActing, planning, RAG, and tool-calling, have all been considered in our modeling of LLM applications. Finally, we admit that it is impossible to exhaustively list all the LLM applications here, but we believe the applications in Fig. 1 are sufficiently representative because they cover the essential characteristics of general LLM applications (as summarized later in Table 1).

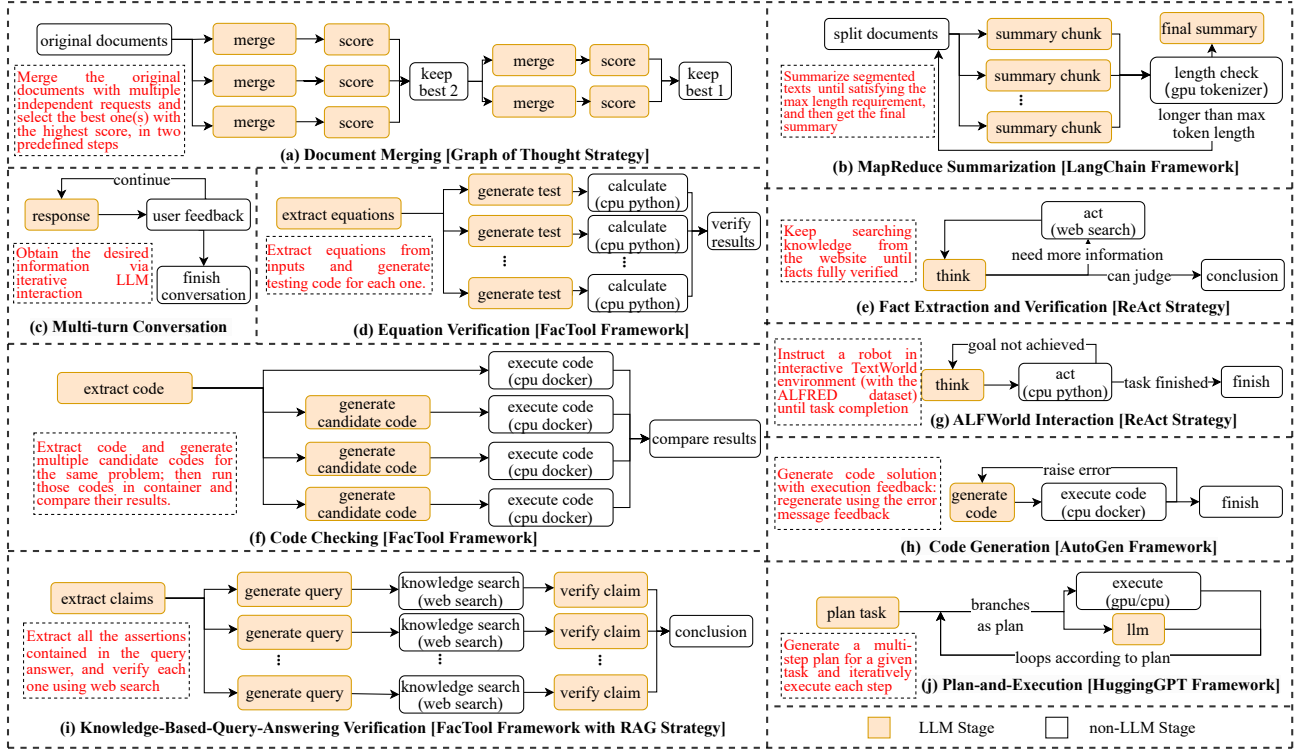


Figure 1. The structures of typical LLM applications (sources listed in Table 1). An application can be built with dedicated frameworks (e.g., FacTool and AutoGen) or following certain control strategies (e.g., Graph of Thought and ReAct).

2.1.2 Challenging characteristics of LLM applications.

The serving quality of LLM applications is critical to user experience and the total service profit. Nonetheless, it is indeed quite challenging to efficiently serve the LLM applications given their distinct characteristics as follows:

1) *Demand Dynamicity*. The resource demand of LLM applications exhibits strong dynamicity on two levels. First, the application structure (i.e., the number and organization of the constituting requests) may be uncertain in advance. Although some applications (like the hard-coded inference workflows with LangChain [69]) have a fixed structure, for some other applications with ReActing [44, 72] or LLM-planning [61], the application structure can only be determined at runtime based on the inference contents. Second, for each individual request, as pointed out by many existing works [35, 71], both the input and output token lengths are dynamic. This implies that the inference duration and peak memory consumption (which are positively correlated with the number of tokens) are also difficult to predict a priori.

2) *Resource Hybridity*. While LLM inferences with GPU backends are the mainstream computations in LLM applications, there also exist various non-LLM tasks in many agent-like applications. Those non-LLM tasks demand additional resources other than GPUs. For example, in the code-generation application provided by the FacTool framework,

the LLM-generated codes need to be tested on CPU backends (e.g., with a docker container [5]). Meanwhile, remote knowledge retrieval [48] would consume network resources. Efficient serving of LLM applications would require coordinated provisioning of such hybrid resources.

3) *Objective Diversity*. Different LLM applications, based on their respective service contents, may have different performance objectives. For example, for many applications where only the final outputs matter—like document-summarization [45] or code-generation [6], users only care about the ultimate application completion time. Meanwhile, applications may also have different service level objectives: for applications whose intermediate results need to be timely returned to users—like conversational text streaming services [52], the performance objective is to attain high instantaneous generation speed (as measured by the *Time-To-First-Token* and *Time-Per-Output-Token* metrics [76]). In some other cases, the performance objective can also be to get the final output prior to the specified deadline [55]. We note that the performance objective is a built-in property of an LLM application, which shall be indiscriminately respected in scheduling.

In Table 1, we check each application in Fig. 1 against the above aspects, which suggests that those characteristics are quite common for modern LLM applications. Note that such characteristics are significantly different from the traditional model serving workloads prior to the LLM era [29, 74] (which

Task	Demand Dynamcity			Resource Hybridity		Objective Diversity		
	Structure	Parallelism	I/O	GPU	Non-LLM backends	ACT	DDL	TPT
(a) DM [7]	✗	✗	✓	✓	✗	✓	✓	✗
(b) MRS [16]	✓	✓	✓	✓	✓ (GPU: tokenizer)	✓	✓	✗
(c) MC [12]	✓	✗	✓	✓	✗	✗	✗	✓
(d) EV [8]	✗	✓	✓	✓	✗	✓	✓	✗
(e) FEV [9]	✓	✗	✓	✓	✓ (Network: search)	✓	✓	✗
(f) CC [5]	✗	✗	✓	✓	✓ (CPU: docker)	✓	✓	✗
(g) ALFWI [4]	✓	✗	✓	✓	✗	✓	✓	✗
(h) CG [6]	✓	✗	✓	✓	✓ (CPU: docker)	✓	✓	✗
(i) KBQAV [10]	✗	✓	✓	✓	✓ (Network: search)	✓	✓	✗
(j) PE [11]	✓	✗	✓	✓	✓ (GPU: small models)	✓	✓	✗

Table 1. Characteristics in aspects of *demand dynamcity*, *resource hybridity* and *objective diversity* for the LLM applications in Fig. 1 (for conciseness, hereafter we use the uppercase letter abbreviation to represent each application). Regarding the objectives, ACT means to reduce Application Completion Time, DDL means to meet a preset completion deadline, and TPT means to guarantee the instantaneous performance on *time per (processed) token* (for simplicity, here we do not differentiate prompt or completion tokens).

have fixed inference time, only consume GPU resources, and share the same performance objective). In a nutshell, to design an ideal serving system for LLM applications, we need to properly deal with such distinct characteristics, seeking to yield high service quality at the application level—which is directly perceived by end users.

2.2 Prior Arts and Their Limitations

In the literature, many research works have been proposed on optimizing LLM inferences in aspects like operator implementation [30, 39], model deployment [20, 34, 46, 47, 76] and inference algorithm [22, 53, 67, 71, 73]. Nonetheless, they mostly focus on individual inference requests, being oblivious to the request correlations and often suffering sub-optimal application-level performance. In particular, when scheduling multiple requests, the default policy adopted in popular inference frameworks like vLLM [17] is First-Come-First-Serve (FCFS) (largely due to demand uncertainty). Fig. 2 shows two LLM applications (A and B) served by a LLM engine (which we assume can serve only one request at a time, i.e., with a batch size of 1). The default request-level FCFS scheduler (Fig. 2a) would interleave the inference serving of different applications, leading to long average Application Completion Time (ACT). However, if we prioritize all the inference requests of B over A (Fig. 2b), we can substantially reduce the average ACT.

Apart from accelerating individual LLM inferences, there do exist some recent works on improving the performance of LLM applications. For example, Parrot [51] employs a simple heuristic that schedules the requests of an application together to optimize the end-to-end performance. However, merely executing the correlated request together is not enough: as shown in Fig. 2, the average ACT when instead

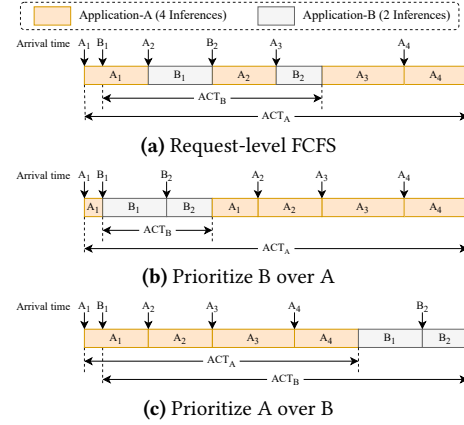


Figure 2. A toy example showing the scheduling algorithms' impact on the overall performance of two LLM applications (A and B, respectively with 4 and 2 dependent LLM requests indexed by the subscripts). Compared with request serving in FCFS (first-come-first-serve) manner, prioritizing all requests of B over A can yield a shorter average ACT (Application Completion time), yet an improper order (prioritizing A over B as in FCFS manner) leads to a worse performance.

prioritizing A over B (Fig. 2c) is even worse than the default case (Fig. 2a). Apart from Parrot, CachedAttention [35] and Infercept [19] focus on improving the effectiveness of KV-cache reusing between dependent requests of an LLM application; Teola [68] applies compilation and batching optimizations to accelerate individual applications; Andes [52] seeks to improve the instantaneous token generation performance for text streaming services.

Nonetheless, although those works are application-aware, they largely ignore the three characteristics of LLM applications—by focusing on a single application structure (e.g., text streaming [52]), a single resource type (e.g., KV-cache [35]) or a single scheduling objective (e.g., end-to-end completion time [51]). There is a lack of a serving system designed for general LLM applications with their common characteristics properly handled, which further leads to two consequences.

First, due to the overwhelming demand dynamcity, it is hard to know the application-level resource consumption, rendering it difficult to improve the overall ACT with advanced scheduling algorithms like SRPT (letting alone to smoothly support diverse scheduling objectives). Second, when serving LLM applications in the dark, it is hard to co-ordinately provision the desired resources on each backend, and a single LLM application—even when it is prioritized in the global queue—may still be impeded on any backend due to factors like cold cache, non-LLM resource contention and improper backend configuration (detailed later in §4.4).

Objective. Motivated by the deficiencies of existing works, our objective in this paper is to leverage request correlations for efficient LLM application serving. In particular, we seek to (1) improve the overall application scheduling performance

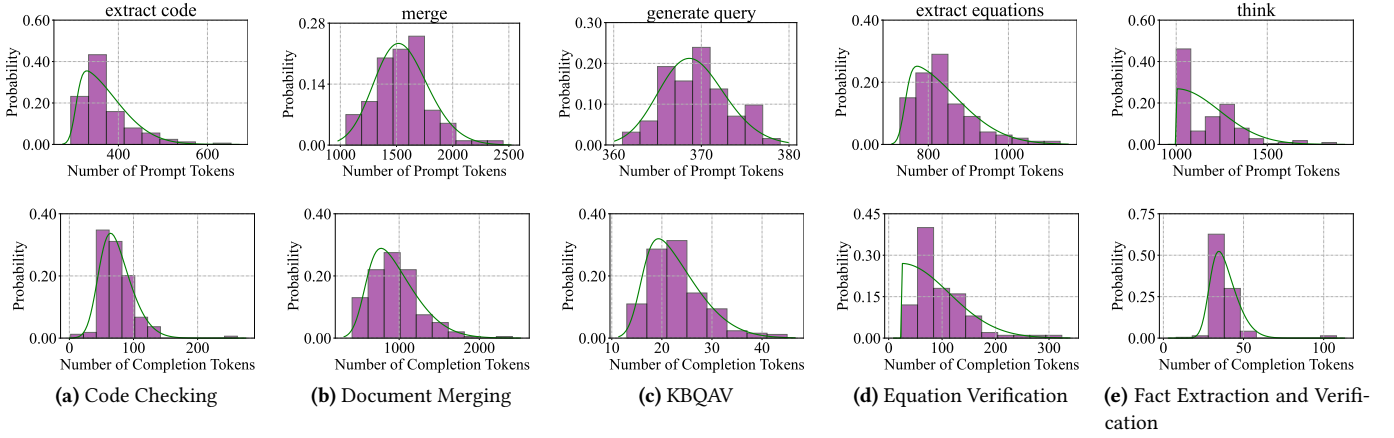


Figure 3. Prompt and completion token length distribution for an arbitrarily-selected request (marked at the top) in five applications each over 100 trial runs. In each case, we divide the length range into 10 buckets, and calculate the value appearance probability in each bucket (accompanied by the fitted curves assuming skewed Gaussian distribution for reference).

involving diverse objectives, and (2) speed up individual LLM applications demanding hybrid resource types.

3 Coinference: The Demand Abstraction for LLM Applications

To facilitate efficient serving and demand modeling of an LLM application, we propose the abstraction of *coinference*, which refers to the entire set of LLM inferences within an LLM application. To better elaborate the concept of coinference, we resort to Fig. 1, which shows the execution workflow of a series of representative LLM applications. As shown in Fig. 1, each application involves both LLM (in orange) and non-LLM tasks (in white). Note that, although an LLM application has hybrid resource demands, our coinference abstraction here sets only the LLM inference requests as first-class citizens: LLM inferences usually take the largest service cost, and meanwhile they are the common parts in all the LLM applications with clear execution patterns (we will show how to integrate the non-LLM resource demands later in §4.2).

Feasibility. We note that the correlations among LLM requests are easy to obtain in practice. For example, when a user executes an LLM application by calling the OpenAI assistants API [3] (essentially a customized cloud-hosted agent), the OpenAI backend associates all the messages (LLM requests) for that application with a common *thread ID*. Moreover, the metadata in LLM requests is often flexible to customize—for example, the `extra_body` parameter is provided by OpenAI [13] and `vllm` [14] to accept additional JSON properties, thus the application affiliation information can be easily integrated into each LLM request.

Opportunities. As aforementioned, formally adopting the coinference abstraction brings two opportunities on serving enhancement for LLM applications. First, as showcased in Fig. 2, by scheduling the LLM requests at the coinference

level (in proper order), we can remarkably improve the overall scheduling performance against diverse application-level objectives. Second, coinference-level performance modeling can help more precisely monitor the demand status of each LLM request, which we here verify empirically.

To elaborate, we run five LLM applications from Fig. 1 each for 100 times (with different inputs from the original dataset). Fig. 3 records the execution statistics in input (prompt) and output (completion) token lengths for an arbitrarily selected request of each application. As shown in Fig. 3, although the resource demands are non-deterministic, they do exhibit salient stability across different trial runs. For example, the output token length of the merge request in Document Merging application is around 1000, yet for the generate-query request in KBQAV application it is between 10 and 50. That phenomenon is indeed reasonable because demand volume is to some extent an inner property based on the usage scenario of an LLM application: the output of the merge request is a document, yet for the generate-query request it is a short query—thus its length distribution has a much smaller mean value and also a much smaller deviation. This way, the historical statistics can help to narrow down the estimation range of request demands and facilitate accurate prediction. Moreover, as we show later (§4.2.2), for requests with dependencies, the upstream requests just-finished can also work as an informative indicator of the demand quantity of downstream requests. This can further strengthen the effectiveness of coinference-based demand prediction.

To summarize, the coinference abstraction—as a scheduling unit as well as a demand predicting approach—makes it quite promising to realize our previous objectives (multi-application scheduling enhancement and single-application processing acceleration). Note that in the network community, there exists a similar notion, *coflow* [27, 28], which seeks to group together all the flows of a data analytics job in scheduling, to make better end-to-end performance. Yet

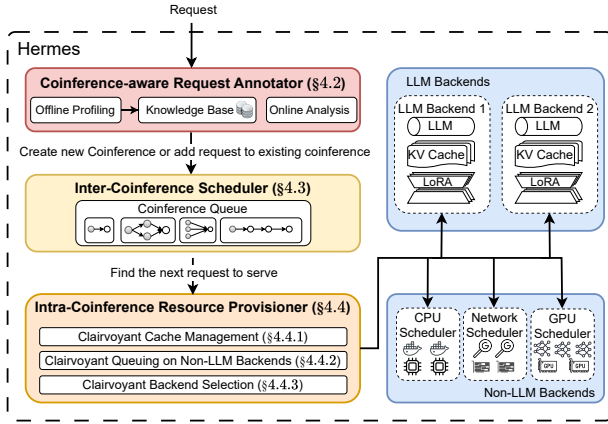


Figure 4. The system architecture of Hermes.

coinference serving is more difficult than coflow serving: for a coflow, the inner structure is simple (in parallel without dependency); the demand quantity (flow size) is often available a priori; the service objective is mostly to minimize coflow completion time; the resource consumption only involves the network backend. In that sense, we are addressing a brand-new problem where the coinference abstraction is only a starting point. Next, we show how to use the coinference abstraction to improve the service quality of LLM applications with their challenging characteristics handled.

4 Hermes: Coinference-Centric LLM Serving

4.1 Overview

Based on the above coinference abstraction, in this paper we design Hermes, a coinference-centric serving system for LLM applications. Hermes has three key modules as shown in Fig. 4. First, there is a *Coinference-aware Request Annotator* (§4.2) that annotates the coinference-level demand information for each incoming request, so as to facilitate later inter- and intra-application optimizations. The demand models are built offline with a series of profiling runs and stored in a knowledge base. Second, the *Inter-Coinference Scheduler* (§4.3) determines the proper scheduling priority of different requests from the coinference-level viewpoint. Third, to efficiently execute each individual request, the *Intra-Coinference Resource Provisioner* (§4.4) leverages the coinference-level demand information to prepare the desired resources in various aspects: cache management (§4.4.1), queuing management on non-LLM backends (§4.4.2), and selection of LLM backends with different configurations (§4.4.3).

4.2 Coinference Demand Modeling

In coinference demand modeling, we seek to panoramically capture the demand characteristics of an LLM application through a series of offline profiling runs. Our objective here is to design a (1) *general* representation method that is applicable to diverse LLM applications and, in the meantime, (2) as *precise* as possible.

```
"application-1":{
  "stage_list":[
    "stage-1":{
      "prompt_tokens": [500, 540,..., 510],
      "completion_tokens": [200, 250,..., 220],
      "request_parallelism": [3, 3,..., 3],
      "loop_times": [2,3,..., 4]
      "next_stage": "stage-2",
      "gap_to_next_stage": [1.2, 1.1, ..., 1.5],
      "demand_during_gap": {
        "cpu_core": 1,
        "memory": 2,
      },
      "demand_correlation_mask": (0,1,1,0,1),
    },
    "stage-2":...,
    "stage-3":...,
    ...
  ],
}
```

Figure 5. An example of coinference demand model.

4.2.1 Towards general demand modeling. We first focus on the *generality* aspect. To design a general demand modeling method, we need to properly handle the *diversity* and *uncertainty* of LLM applications. We will respectively show our solutions in such two aspects with Fig. 5.

1) *Handling structure and resource diversity.* In typical LLM applications, there are heterogeneous relationships among the requests of a coinference: some requests are in parallel with each other, whereas some others have rigid dependencies. To describe such diverse request relationships in a general manner, we adopt a *stage-centric* paradigm. An LLM application usually contains a series of functional units—each issuing one or multiple parallel inferences that are submitted almost concurrently and also share the *same system prompt*; such a functional unit is defined as a *stage*². Therefore, we can organize the requests of a LLM application as a list of stages; each stage records the request-level demand (input/output token length) and also maintains a *next_stage* property to record the dependency information. Second, executing an LLM application also requires diverse resource backends like CPU and network (for tool calling); such non-LLM resources are usually consumed during the gaps between LLM inferences. To express such non-LLM resource demand, we incorporate the consumption amount (e.g., 1 cpu core and 2GB memory) as well as the gap length to the dependency information, as expressed by the *gap_to_next_stage*³ and *demand_during_gap* properties in Fig. 5.

2) *Handling structure and request-level uncertainty.* Demand uncertainty (i.e., runtime dynamicity) means that an LLM application has different stage structures or demand

²The stage affiliation of a request can be manually annotated by the `extra_` body parameter (as the application affiliation) or automatically detected by analyzing the prompt prefix.

³For LLM applications without tool calling (e.g., for Chatbot), `gap_to_next_stage` can also be leveraged to represent the typical interaction pace incurred by factors like network transmission delay.

quantities in different execution rounds. Structural uncertainty is mainly caused by the looping LLM requests in ReActing-like applications⁴. To build a stable application structure despite such looping dynamicity, we introduce the `loop_times` property in each stage to describe its looping status across different profiling rounds. Moreover, when expressing the quantity variation in each demand aspect (input/output token length, request parallelism, loop times and post-stage gap), we choose to directly append the raw value profiled from each trial run to a list. We note that recording the raw values is better than recording the coefficients of the fitted skewed norm distribution, considering the depiction fidelity (the true distribution is in fact irregular) and computation efficiency—it takes much time (up to seconds) to fit out the coefficients as well as to support our later calculations (e.g., correlation analysis and integral calculus). In practice, we set the maximal length of the recorded value list to 500 (evicted in FIFO manner), and the storage cost is indeed negligible on cutting-edge GPU servers.

4.2.2 Towards precise demand modeling. Note that the ultimate purpose of demand modeling is to support accurate demand prediction at runtime. So far, we have made that by creating a general stage demand portrait from the past execution rounds. Apart from such cross-round hints, we can future enhance the prediction accuracy by correlation hints in the ongoing round. Due to the structural dependencies among stages, the resource demands of a stage are often correlated to the upstream ones. We note that there typically exist three cross-stage⁵ demand correlation patterns:

- 1) A stage's request input length may be correlated to the upstream stage's request input length and output length. For example, for the Document Merging application, the input of each request in the *scoring* stage is a superset of a request's output in the (upstream) *aggregate* stage (plus a fixed system prompt). Meanwhile, for the looping stages, downstream requests would share the same prompt template as the upstream ones, indicating similarity in input length.
- 2) A stage's request output length may be correlated to its input length as well as the upstream stage's request output length. For example, for the *generate-code* stage in Code Checking application, a more complex input task (often with a longer prompt) usually yields a longer code segment; so for other LLM-tasks like translation and article correction/polishing. Meanwhile, for the looping states, the request output lengths are also similar across stages.
- 3) A stage's request parallelism may be correlated to the upstream stage's request parallelism.

⁴We admit that plan-and-execute applications may also yield dynamic structures, which is difficult to address given the diverse functional units after planning. For consistent modeling with reasonably good performance, we dictate that each plan-and-execute application always has two stages—a *planning* stage and an *executing* stage. In that sense, a N-step-after-planning application would have an *executing* stage that loops for N times.

⁵For simplicity, we assume a Markov model in correlation analysis and only consider the correlation between directly dependent stages.

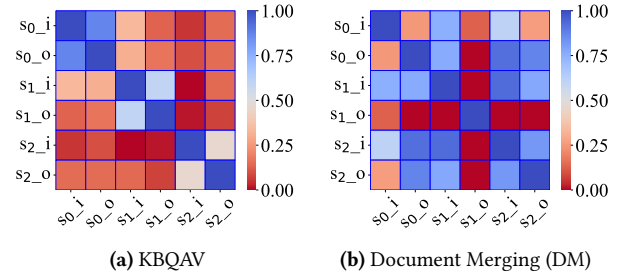


Figure 6. The Pearson correlation coefficients between input/output token lengths of different stages. We present the first three stages of KBQAV and DM, where s_{x-i} and s_{x-o} represent the input and output token length of the x -th stage.

For example, as shown in KBQAV applications, for each inference request in the (upstream) *generate-queries* stage, a corresponding inference request would be launched at the (downstream) *verify-claim* stage.

To verify the existence of above correlation types (especially the first two), we resort to Pearson Correlation Analysis [60] based on our profiled data over 100 trial runs. We also divide the demand range to 10 buckets (as in Fig. 3 for ease of analysis), and use $P(X = i)(i = 0, 1, \dots, 9)$ to represent the probability that the demand quantity variable X falls into bucket- i . Then we calculate the Pearson correlation coefficient between two demand variables X and Y as $\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{\mathbb{E}((X-\mu_X)(Y-\mu_Y))}{\sigma_X \sigma_Y}$. In Fig. 6, for KBQAV and Document Merging, we further depict the Pearson correlation coefficients between any two demand variables (each represent the input/output length of a stage). For KBQAV, each stage's output length is strongly correlated to its input length; yet for Document Merging, each stage's input length is highly-related to the upstream stage's input length. Fig. 6 also suggests that each application has distinct correlation patterns; for prediction efficiency, we only consider the demand correlations with a coefficient (ρ) larger than 0.5. To that end, in each stage we add a five-tuple $(M_I^I, M_I^O, M_O^I, M_O^O, M_P^P)$ to mask whether the corresponded demand correlation holds: M_B^A represents whether the demand variable A affects B ; I , O and P ($\tilde{I}$, $\tilde{O}$ and $\tilde{P}$) respectively represents the input length, output length and request parallelism of the current (upstream) stage.

The above correlation analysis enables more precise online demand prediction. Upon the completion of a stage, its execution information can be immediately adopted for demand prediction of the next stage. In that case, we are facing a *conditional prediction* problem. Suppose the request input and output length of the just-finished stage is respectively in bucket i and o —with both correlated with the input length of the downstream stage, and we need to predict $P(I|\tilde{I} = I, \tilde{O} = o)$. To accomplish that, we join the historical execution records of the two stages (i.e., yielding $(\tilde{I}, \tilde{O}, I)$ tuples respectively from each trial round), and filter out the

profiled tuples satisfying $\tilde{I} = i$ and $\tilde{O} = o$; then the I distribution within the filtered records is a more precise prediction for the current round.

4.3 Inter-Coinference Scheduling

As previously showed in Fig. 2, the conference abstraction provides us with a high-level scheduling handle. In this part, we explore how to leverage the coinference modeling results to optimize the inter-coinference scheduling performance. There are two specific challenges. First, how to leverage the distribution-style demand description for scheduling efficiency optimization? Second, how to deal with the cases where the competing LLM applications have heterogeneous service objectives? We next address them respectively.

4.3.1 Efficient coinference scheduling with Gittins policy. We first focus on optimizing the average application completion time. A classical algorithm proven optimal in that regard is shortest-remaining-processing-time (SRPT) [58], which sets the priority of each request based on the remaining processing time of its coinference. However, naively applying SRPT is not appropriate for our problem: SRPT relies on deterministic demand values to make scheduling decisions, yet the resource demands of LLM applications in various aspects are usually uncertain a priori (our prior modeling strategies only mitigate but not eliminate such uncertainties). Although it is possible to use the expected value (mean) of a demand distribution to emulate SRPT scheduling, this method often fails to work well due to its blindness to instantaneous execution progress. For example, for a request with an expected output length of 20 tokens, it is possible that its token generation process does not stop even after 100 tokens. In that case, we need to timely refresh the demand estimation based on the latest progress rather than sticking to the prior expectations (otherwise the remaining processing time—the expected execution time minus the executed time—may ironically become negative). Therefore, we need to introduce uncertainty-awareness in designing our scheduling algorithm.

The problem we now face—minimizing the average completion time for jobs with *unknown durations* but *known duration distributions*—has been well studied in the literature, for which the *Gittins* policy [18, 36, 59] has been proven optimal. The basic idea of Gittins policy is to calculate a *Gittins rank* for each job—based on its executed time so far and its size distribution—as the scheduling priority. Specifically, let a request's output length distribution be $\mathcal{D}$, and suppose a tokens have already been generated, then its Gittins rank G can be expressed as:

$$G(\mathcal{D}, a) = \inf_{\Delta > 0} \frac{\mathbb{E}[\min\{X_{\mathcal{D}} - a, \Delta\} \mid X_{\mathcal{D}} > a]}{\mathbb{P}\{X_{\mathcal{D}} - a \leq \Delta \mid X_{\mathcal{D}} > a\}}, \quad (1)$$

where $X_{\mathcal{D}}$ is the random variable under $\mathcal{D}$. Given a service budget Δ (i.e., the number of additional tokens allowed to be

generated henceforth), the denominator in Eq. 1 represents the possibility that the generation process can finish before Δ (which contributes to small average completion time), and the numerator represents the corresponded resource consumption (the cost paid to serve it). Note that G is always no larger than the maximum value in $\mathcal{D}$, and it can be viewed as the true runtime estimator of the remaining processing demand when determining the scheduling priority: existing works [36, 59] have shown that this method can theoretically yield the minimal average completion time.

So far, we apply the Gittins policy on estimating the effective request output length at runtime, yet our ultimate target here is to estimating the remaining processing time of the entire coinference. To that end, we apply Gittins policy also to the estimation of remaining stage looping times. That is, after executing a looping stage, we re-estimate its remaining looping times (at bucket granularity) also with the distribution of its looping times as well as the times it has already looped for. Regarding the other demand types, since they are submitted not incrementally but in one batch (input token length and stage parallelism), or are simply not token-based (non-LLM demands), we choose to still use the mean value.

Therefore, to realize SRPT for LLM applications, for an application with totally N stages, we estimate its total execution demand as $T = \sum_{j=1}^N n_{\text{loops}}^j * n_{\text{parallelism}}^j * t_{\text{request}}^j + t_{\text{gap}}^j$, where n_{loops}^j , $n_{\text{parallelism}}^j$ and t_{gap}^j are respectively the estimated looping times, request parallelism and post-stage gap for stage j , and $t_{\text{request}}^j = \bar{t}_{\text{prompt}} * n_{\text{prefill}}^j + \bar{t}_{\text{decode}} * n_{\text{generation}}^j$ is the per-request duration. Here $\bar{t}_{\text{prefill}}$ ($\bar{t}_{\text{decode}}$) is the average processing time of each prefilling (decoding) token, both relatively stable and measured at runtime.

4.3.2 Integration of diverse coinference scheduling objectives. Recall that, apart from the ultimate completion time, LLM applications may also care about other service-level objectives like deadline (i.e., completion time no later than a target value) or instantaneous service speed (i.e., time-per-token no larger than a target value). We need to smoothly deal with such heterogeneous objectives in scheduling. In principle, we try minimize the average application completion time while satisfying the application-claimed SLO requirements. To that end, we set SRPT as the default scheduling algorithm for all applications; for applications with SLO requirements, we intervene only when the risk emerges that their SLO targets would be violated under the default SRPT algorithm. Therefore, we need to continuous monitor such SLO-violation risk levels.

For applications requiring guaranteed time-per-token (TPT) performance (specified as TPT_{SLO}), we monitor the TPT-violation risk, r_{TPT} , as: $r_{\text{TPT}} = \frac{t_s/n_o}{\text{TPT}_{\text{SLO}}}$, where t_s is the application service time since its submission, and n_o is the accumulated number of generated tokens for that application. When r_{TPT} exceeds a specified threshold (defaulted to

0.8), the coinference is marked with a *TPT-risky* flag and directly set to highest scheduling priority. To avoid frequent context switching, once a coinference is marked as *TPT-risky*, it is forced to execute for a minimum number of iterations (100 rounds by default) before clearing the *TPT-risky* flag and regaining the scheduling priority under SRPT.

For applications requiring guaranteed service completion time before the deadline, we monitor the deadline-violation risk, r_{DDL} , as $r_{DDL} = \frac{\text{worst_case_remaining_processing_time}}{ddl - \text{now}}$, where *worst_case_remaining_processing_time* is calculated based the longest execution time observed in historical runs. If r_{DDL} exceeds a preset threshold (also defaulted to 0.8), the application will be moved to the front of the queue (yet still after the TPT-prioritized tasks because TPT requirements are less elastic). This way, we can concurrently meet the three objectives for LLM applications, yielding small average ACT while providing SLO guarantees.

4.4 Intra-Coinference Resource Provisioning

We previously focus on inter-coinference scheduling, and the essential is to iteratively determining the coinference that should be served with the highest priority (for any scheduling objective). However, we note that the highest priority itself does not necessarily yield fast coinference completion: as elaborated in §2, serving a coinference often involves hybrid (LLM and non-LLM) resource types from multiple backends. For a prioritized coinference, although it would not be delayed by other coinferences, it could still be delayed by the late provisioning of the desired resource backends. In this part, to facilitate the fast completion of individual coinferences, we propose *coinference-aware clairvoyant resource provisioning*, which works in three aspects.

4.4.1 Clairvoyant Cache Management. Caching is significant for efficient LLM serving. First, existing works [19, 47] have shown that KV-cache can remarkably improve the token generation speed, which may even need to be maintained across different inference requests (e.g., for multi-turn conversations [35]). Failing to cache such tokens—meaning that they must be either recomputed or loaded from lower-level storage (i.e., from CPU to GPU or from disk to CPU)—would result in large time overhead. Our measurements with an A100 GPU server show that, given 1000 tokens, the time to recompute the KV-cache or loading them from CPU memory to GPU HBM (from SSD to CPU memory) is respectively around 92ms or 35ms (5,627ms). Second, Low-Rank Adaptation (LoRA) [41] is prevalently adopted to support LLM serving with customized models, which, in production clusters with diverse LoRA adaptors, requires runtime adaptor loading to multiplex the base model [63]. Loading typical LoRA adaptors may also take non-negligible time in the critical path [49]. Therefore, we need to properly manage the shared caching space for both KV-cache and LoRA adaptors.

With the coinference abstraction, we then propose clairvoyant cache management. First, for cache eviction due to

insufficient cache space, rather than adopting the LRU policy (default under vLLM [47]), we propose to evict the cached items based on the priority of the coinference they belong to: the higher the coinference priority, the higher the cache priority. This way, we can avoid evicting the KV-cache or LoRA adaptors of high-priority coinferences during their inter-stage gaps. Second, in cases where a high-priority coinference yields its cache space to others during its relatively long inter-stage gaps (e.g., waiting for tool feedback or future user inputs), for its fast execution, we need to prefetch its KV-cache or LoRA adaptors prior to the arrival of its next stage. In that regard, too-early prefetching would waste the memory resources, whereas too-late prefetching would delay the completion of that high-priority coinference. To determine the appropriate time to trigger prefetching, Hermes leverages the profiled gap distribution recorded in the *gap.to.next*–stage property. To be specific, suppose a stage completes at time t_0 , and the confidence interval of the following gap duration (under the significance level of $\alpha = 0.05$) is $[d_{\min}, d_{\max}]$. Then Hermes triggers prefetching at time $t_0 + d_{\min} - t_p$, where t_p is the time taken by the cache prefetching operation (we do consider layer-wise prefetching [35] in implementation). Moreover, if $d_{\min} - t_p$ is negative, we simply preserve the KV-cache or LoRA adaptors without swapping.

4.4.2 Clairvoyant Queuing on Non-LLM Backends.

Serving LLM applications often requires diverse non-LLM backends, the demands of which, as elaborated in §4.2, are included in coinference modeling. In production clusters concurrently serving massive LLM applications, it is possible that resource contention occurs on such non-LLM backends. To elaborate, we focus on three typical types of LLM tools⁶ (here we use a broad sense of *tools* which refer to all the non-LLM components of LLM applications). First, some code-writing applications [5] may periodically verify the correctness of the generated code with a docker container (for better environment isolation); under insufficient CPU resources, the container launching requests would be queued on the scheduler. Second, LLM applications with RAG (like KBQAV [10]) may periodically search for documents from the websites, which, under limited network bandwidth, may cause non-negligible network queuing delay. Third, LLM applications like HuggingGPT [11] may call small models (hosted on dedicated GPU servers); facing bursting requests, such small models (constrained by the resources on the dedicated GPU servers) can also become a bottleneck.

Note that our previous coinference scheduling priority is imposed to the LLM backends, and the above non-LLM tasks submitted to other backends are often handled by independent schedulers. For example, the docker launching

⁶While we pick up three relatively-simple tools for study in this part, we note that the general principle here can be applied to other professional tools, like the traditional symbolic solver used in AlphaGeometry [1].

requests would be submitted to a kubernetes scheduler. Under the default strategy (e.g., FCFS) of those schedulers, it is possible that the non-LLM tasks of a high-priority application be queued for a long time, postponing the ultimate application completion. To address this issue, we propose *clairvoyant queuing on non-LLM backends*, which propagates the application priority to all the involved schedulers managing non-LLM backends. This way, it can be ensured that both the LLM and non-LLM components of a high-priority application can be serviced without any queuing delay.

4.4.3 Clairvoyant Backend Selection. So far, we focus on the cases with uniform LLM backends. In practice, it is possible that the LLM service providers provision multiple backends each with distinct configurations [51]. For example, inference batch size is a critical hyperparameter that affects the latency-throughput trade-off: setting a larger batching size can yield a higher throughput (by 8.2 $\times$) but the latency would also be higher (by 95%) [23]. For fast serving of LLM applications, we thus also need to properly select the backend for each stage based on its demand characteristics.

To address that problem, we can employ a heuristic, clairvoyant backend selection, which leverages our coinference demand information to judge backend suitability for each stage. Suppose there is a latency-sensitive backend (with batch size $B_{\min}$) and a throughput-sensitive backend to choose—emulating a “power-of-two-choices” scheduling space [64]; given a stage, we schedule it to the latency-sensitive backend if its request parallelism is larger than $B_{\min}$, and otherwise to the throughput-sensitive backend, so as to attain fast stage completion. Our evaluations show that this heuristic can indeed work well in practice. We note that the principle of clairvoyant backend selection can be extended to other multi-backend cases. For example, a cluster may have GPU nodes of different generations (with heterogeneous computation-memory ratios); based on the predicted request output length, we can schedule the (memory-intensive) long-output requests onto the backends with smaller computation-memory ratio (and vice versa), which can improve the overall resource utilization.

5 Evaluation

Implementations. We have implemented Hermes atop vLLM with over 4,100 lines of Python codes. The functionalities of CoInference-aware Request Annotator in Fig. 4 are implemented in CoinferenceAnnotator, and we replace the default vLLM scheduler to our CoinferenceScheduler. Regarding intra-application resource provisioning, the CoinferenceScheduler passes the queue information to a series of lower-level resource managers—BlockManager and LoRAManager for cache management, DockerManager, RetrievalManager and ModelManager for non-LLM resource management, and ServerSelector for LLM backend selection. Additionally, we build a comprehensive benchmark

suite consisting of 11,300 lines of Python code, which includes all of the ten application types depicted in Fig. 1.

Hardware Platform. We evaluate Hermes respectively with two different setups of single-GPU and multi-GPU experiments. The large language models we use are LLaMA-7B and LLaMA-13B. The single-GPU evaluations use a server with four 16-core AMD-EPYC-7302 CPUs, 128GB RAM, 4TB SSD and one NVIDIA A100-PCIE-40GB GPU. The multi-GPU evaluations use a server with four 20-core Intel-Xeon-Gold-6133 CPUs, 258GB RAM, 2TB SSD and four Tesla V100-PCIE-16GB GPUs. Both servers run CUDA 12.4.

Workloads. For our experiments, we create a mixed workload suite with totally 500 LLM applications (each with distinct inputs from the original datasets). In that workload suite, 20% applications are multi-turn conversations requiring guaranteed TPT performance (TPT-sensitive), 20% require meeting a preset deadline (DDL-sensitive), and the remaining 60% target for short ACT (ACT-sensitive). To get the applications not involving TPT objectives, we randomly sample from the 9 non-conversation application families in Fig. 1. In particular, similar to prior work [42, 56, 75], we set the sampling probability of small (EV, FEV, CC, ALFWI and KBQAV—usually less than 1 min), medium (CG and PE—usually between 1 and 10 min) and large (DM and MRS—usually longer than 10 min) applications to be 72%, 26%, and 2%, respectively. Meanwhile, for the TPT-sensitive applications, we set the TPT target as 0.1 s/token (given that typical human reading speed is less than 600 token per minute [2]). For the DDL-sensitive applications, we set the deadlines by multiplying the original execution length by a random factor between 1.2 and 2, resembling some previous studies [37, 54]. Regarding application submission, given a submission window of T , we let the submission interval follow a Poisson distribution with the expectation be T/N (N is the total application number submitted in experiments).

5.1 End-to-end Scheduling Performance

Setups. We first evaluate the scheduling performance of Hermes. We run an LLM inference engine on a single A100 GPU as the backend server, and submit all the 500 applications with the submission window respectively set to 10, 15 and 30 minutes (i.e., the workload intensity respectively be 3 $\times$, 2 $\times$ and 1 $\times$). We compare Hermes with three scheduling strategies: Request-FCFS, Coinference-FCFS, and Hermes-ACT. Request-FCFS schedules the request in the First-Come-First-Serve manner, which is the default scheduling policy in mainstream LLM serving frameworks like vLLM [17]. Coinference-FCFS performs FCFS scheduling at the application level (as adopted in Parrot [51]), meaning that each request is scheduled based on the arrival time of the coinference it belongs to. Hermes-ACT is a variant of Hermes which focuses only on minimizing average ACT. Regarding the metrics, for ACT-sensitive applications we measure the

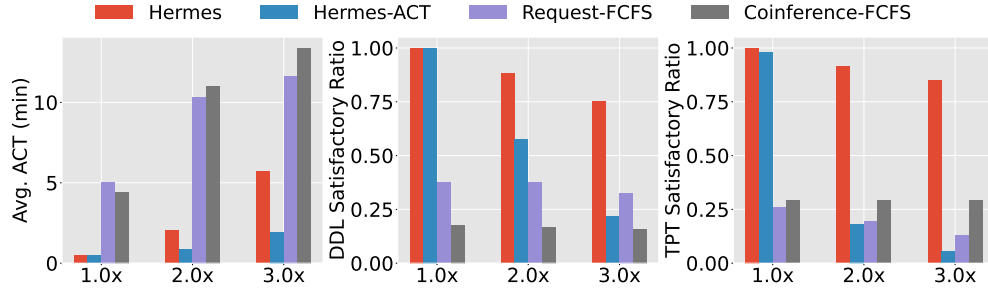


Figure 7. End-to-end LLM application performance of different schedulers against diverse service objectives (under different application arrival intensities).

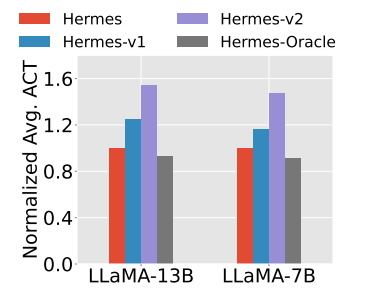


Figure 8. Ablation study on the methods tackling demand dynamicity.

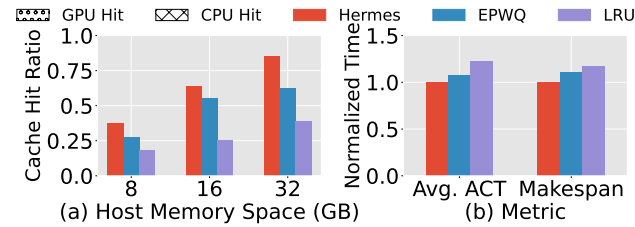


Figure 9. Cache hit ratio (with different host memory sizes) and execution efficiency (measured with both average ACT and makespan) under Hermes, EPWQ and LRU.

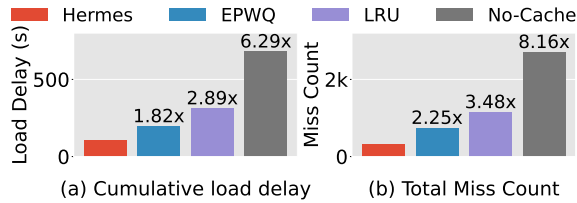


Figure 10. Cumulative load delay and total miss count to load LoRA adapters from disk to memory.

average ACT; for DDL-sensitive applications we measure the DDL Satisfactory Ratio, which is the proportion of DDL-sensitive applications that successfully meet their deadlines; for TPT-sensitive applications, we measure the TPT Satisfactory Ratio, which is the average proportion of checking points (once for every 10s) at which the TPT performance during the preceding interval (10s) satisfies the requirement.

Results. As shown in Fig. 7, Hermes can perform much better than Request-FCFS and Coinference-FCFS in each case. For example, when workload intensity is 1x, the ACT performance of Hermes is 90.1% (88.3%) better than Request-FCFS (Coinference-FCFS). Meanwhile, by comparing the performance of Hermes and Hermes-ACT, we can learn that multi-objective compatibility is indeed quite necessary—especially in cases with intense contention: when the workload intensity is 3x, Hermes-ACT remarkably sacrifices the SLO performance in both DDL and TPT (by respectively 71.2% and 93.6% if compared with Hermes). Additionally, we also take notice of the scheduling overhead of Hermes, which remains

below 1 ms in our measurements of each case, negligible compared to the time-consuming LLM computations.

5.2 Ablation Studies on Tackling Demand Dynamicity

Setup. Recall that to handle demand dynamicity, we have adopted both correlation analysis (§4.2.2) and the Gittins policy (§4.3.1). To verify their effectiveness, we resort to a series of ablation experiments. We first disable the correlation analysis module and get *Hermes-v1*. Then we disable the Gittins policy (i.e., using the mean value of a distribution in SPRT scheduling), and further get *Hermes-v2*. For reference, we also add an idealized baseline, *Hermes-Oracle*, which knows exactly the resource demand of each LLM application (by conducting a trial run with temperature set to 0). In our experiments, we conduct two tests using LLaMA-7B and LLaMA-13B, respectively. LLaMA-7B uses two V100 GPUs with tensor parallel deployment, and LLaMA-13B uses four. We submit 300 applications to the V100 server within a time window of 10 minutes and measure the average ACT (normalized by that under standard Hermes).

Results. As shown in Fig. 8, without correlation analysis, the average ACT would be inflated by 15.3% with the LLaMA-7B model. When the Gittins policy is further disabled, that performance degradation would increase to 47.5%. Such results confirm the indispensability of both the correlation analysis and the Gittins policy in tackling demand uncertainty. Meanwhile, the performance gap between Hermes and Hermes-Oracle is less than 10%, suggesting that Hermes can deliver near-optimal scheduling performance for LLM applications. We also notice that in each case, the performance of LLaMA-7B and LLaMA-13B are quite similar.

5.3 Effectiveness of Clairvoyant Cache Management

KV cache management. To evaluate the effectiveness of Hermes in KV cache management, we submit 500 applications to the A100 server within 15 minutes. We enable the KV cache to be evicted from GPU DRAM to CPU memory and also from the CPU memory to disk (whenever a layer is full), and loaded vice versa. In particular, we introduce two baselines: LRU and Evict/Prefetch-on-Waiting-Queue (EPWQ).

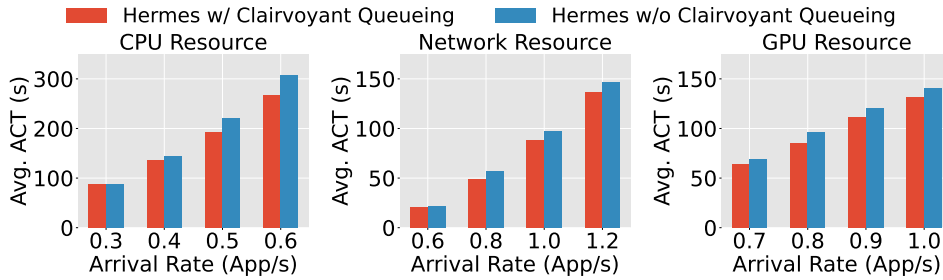


Figure 11. Performance with and without clairvoyant queuing respectively on three different non-LLM backends in cases with resource contention.

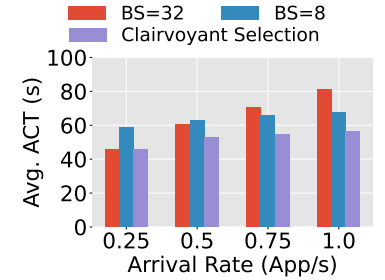


Figure 12. Performance under different backend selection methods.

EPWQ means to prioritize the KV cache based on the pending request queue (adopted in CachedAttention [35]); yet, during the inter-stage gap of a coinference, since the downstream request is not in the queue, its desired KV cache would be of the lowest priority. Under each method, we measure the total cache hit ratio (including both GPU and CPU cache hits) against different CPU memory sizes (8GB, 16GB and 32GB). As shown in Fig. 9(a), Hermes consistently achieves the highest cache efficiency in each case, improving the overall cache hit ratio by up to $1.11\times$ ($0.33\times$) compared to LRU (EPWQ). Fig. 9(b) further depicts the average ACT and makespan under each scheme in the case with 32GB CPU memory. It shows that Hermes can reduce the average ACT by 18% (6%) compared to LRU (EPWQ).

LoRA cache management. To evaluate Hermes performance in LoRA caching, we compare Hermes with EPWQ, LRU, and No-Cache (i.e., not storing LoRA in host memory, which is also the default setting of vLLM). We set `max-loras` (the maximum number of parallel processes per iteration) to 10, `max-cpu-loras` (the number of LoRA caches that can be stored in CPU memory) to 20, and use totally 200 LoRA adapters for LLaMA 7B with a rank of 8. We submit a 25-minute workload containing 1,000 applications, with each application randomly assigned to one of the LoRA adapters (all requests of an application use the same LoRA). Fig. 10 shows the load latency and miss count under each scheme. According to Fig. 10, compared to the second best (EPWQ), Hermes can reduce the LoRA load latency by 45%, and reduce the number of LoRA misses by 55.2%.

5.4 Effect of Clairvoyant Queueing on Non-LLM Backends

To assess the effectiveness of clairvoyant queuing on non-LLM backends (§4.4.2), we select three LLM applications, CG, KBQAV and PE, which respectively demands docker (to test code in a virtual environment), network (to search knowledge from the web) and GPU (to run the small models) resources. The test environment contains two V100 (16G) GPUs serving the LLaMA-7B model. We build a prototype where each of the above resource backends is managed by an independent scheduler with limited service concurrency (for simplicity we manually set such limitation). For Docker

backend, we limit the total number of CPU cores to 40, with each Docker container allocated 2 cores; for the network backend, we restrict the remote searching concurrency to 4; for the GPU backend (for small models), we limit the maximal concurrency to 15%. In each case, we submit the selected applications under different arrival rates, and Fig.11 depicts the average ACT. Compared with the default case where each backend queues the request independently in FCFS order, Hermes can reduce the average ACT by up to 13%.

5.5 Effectiveness of Clairvoyant Backend Selection

We further evaluate the effectiveness of our automatic backend selection method. To that end, we deploy two LLaMA-7B backends with four V100 GPUs (each model deployed with two GPUs): the batch size for one backend is configured to 32 for high throughput, and the other to 8 for low latency. The schemes compared are our clairvoyant backend selection strategy (i.e., try using throughput-first backend for stages with parallelism larger than 8, and try using the latency-first backend otherwise) and two naïve strategy that always choose a fixed backend (either 8 or 32). We submit 500 LLM applications for multiple runs (in different runs we use different schemes but the application submission trajectory are fixed), and study the average ACT of those applications with non-trivial stage parallelisms (i.e., larger than 1). Such experiments are repeated under different application arrival rates. As shown in Fig. 12, in any case, our clairvoyant backend selection strategy can always make the best performance.

6 Conclusion

In this paper, we propose Hermes, an efficient serving system for LLM applications based on the coinference abstraction. Hermes models a coinference as a list of stages, and depicts the dynamic resource demand in each aspect as a distribution. It further adopts the Gittins policy to optimize inter-coinference scheduling performance supporting diverse objectives. For fast execution of each LLM application, Hermes adopts clairvoyant resource provisioning based on the profiled coinference demand on each backend. Experimental results on popular workloads show that Hermes can remarkably improve the serving efficiency of LLM applications.

References

- [1] 2024. AlphaGeometry: An Olympiad-level AI system for geometry. <https://deepmind.google/discover/blog/alphageometry-an-olympiad-level-ai-system-for-geometry/>. [Online; accessed 19-Oct-2024].
- [2] 2024. Average human reading speed. <https://scholarwithin.com/average-reading-speed>. [Online; accessed 19-Oct-2024].
- [3] 2024. OpenAI Assistants API. <https://platform.openai.com/docs/assistants/overview>. [Online; accessed 19-July-2024].
- [4] 2024. The origin code of ALFWorld Interaction. <https://github.com/ysymyth/ReAct/blob/6bdb3a1fd38b8188fc7ba4102969fe483df8fcd9/alfworld.ipynb>. [Online; accessed 19-Oct-2024].
- [5] 2024. The origin code of Code Checking. <https://github.com/GAIR-NLP/factool/tree/3f3914bc090b644be044b7e0005113c135d8b20f/factool/code>. [Online; accessed 19-Oct-2024].
- [6] 2024. The origin code of Code Generation. <https://github.com/microsoft/autogen/tree/0560bdd645dfbc579a71f20fea98ea83dd3bb3f?tab=readme-ov-file#quickstart>. [Online; accessed 19-Oct-2024].
- [7] 2024. The origin code of Document Merging. https://github.com/spcl/graph-of-thoughts/tree/a939a4577c07c80b8ecb194793b5a4169d99b31b/examples/doc_merge. [Online; accessed 19-Oct-2024].
- [8] 2024. The origin code of Equation Verification. <https://github.com/GAIR-NLP/factool/tree/3f3914bc090b644be044b7e0005113c135d8b20f/factool/math>. [Online; accessed 19-Oct-2024].
- [9] 2024. The origin code of Fact Extraction and Verification. <https://github.com/ysymyth/ReAct/blob/6bdb3a1fd38b8188fc7ba4102969fe483df8fcd9/FEVER.ipynb>. [Online; accessed 19-Oct-2024].
- [10] 2024. The origin code of Knowledge-Based-Query-Answering Verification. https://github.com/GAIR-NLP/factool/tree/3f3914bc090b644be044b7e0005113c135d8b20f/factool/knowledge_qa. [Online; accessed 19-Oct-2024].
- [11] 2024. The origin code of Plan-and-Execution. <https://github.com/microsoft/JARVIS/tree/c62e0faac76c4a2907cabe2cfe4bbe5f2e613400>. [Online; accessed 19-Oct-2024].
- [12] 2024. The origin source of ShareGPT dataset used in Multi-turn Conversation. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered. [Online; accessed 19-Oct-2024].
- [13] 2024. The extra_body parameter supported by OpenAI API. <https://github.com/openai/openai-python/blob/af8e0ad8b7988e72e1d02478022d80eff4f55b91/src/openai/resources/models.py#L51>. [Online; accessed 19-Oct-2024].
- [14] 2024. The extra_body parameter supported by vLLM. https://docs.vllm.ai/en/v0.6.0/serving/openai_compatible_server.html#extra-parameters. [Online; accessed 19-Oct-2024].
- [15] 2024. The Shift from Models to Compound AI Systems. <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>. [Online; accessed 21-September-2024].
- [16] 2024. The tutorials of how to implement MapReduce Summarization. <https://python.langchain.com/v0.2/docs/tutorials/summarization/#go-deeper-1>. [Online; accessed 19-Oct-2024].
- [17] 2024. vLLM: Easy, fast, and cheap LLM serving for everyone. <https://docs.vllm.ai/en/stable/>. [Online; accessed 19-July-2024].
- [18] Samuli Aalto, Urtzi Ayesta, and Rhonda Righter. 2009. On the Gittins index in the M/G/1 queue. *Queueing Systems* 63 (2009), 437–458.
- [19] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. 2024. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. In *Forty-first International Conference on Machine Learning*.
- [20] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *USENIX OSDI*.
- [21] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [22] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774* (2024).
- [23] Lequn Chen. [n.d.]. Dissecting batching effects in gpt inference. <https://le.qun.ch/en/blog/2023/05/13/transformer-batching/>. [Online; accessed 19-October-2024].
- [24] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [25] Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. 2023. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595* (2023).
- [26] I Chern, Steffi Chern, Shiqi Chen, Weizhe Yuan, Kehua Feng, Chunting Zhou, Junxian He, Graham Neubig, Pengfei Liu, et al. 2023. FacTool: Factuality Detection in Generative AI—A Tool Augmented Framework for Multi-Task and Multi-Domain Scenarios. *arXiv preprint arXiv:2307.13528* (2023).
- [27] Mosharaf Chowdhury and Ion Stoica. 2012. Coflow: A networking abstraction for cluster applications. In *Proceedings of the 11th ACM Workshop on Hot Topics in Networks*. 31–36.
- [28] Mosharaf Chowdhury, Yuan Zhong, and Ion Stoica. 2014. Efficient coflow scheduling with varys. In *Proceedings of the 2014 ACM conference on SIGCOMM*. 443–454.
- [29] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. 2017. Clipper: A {Low-Latency} online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 613–627.
- [30] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.
- [31] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [32] Mohamed Fezari and Ahmed Al-Dahoud Ali-Al-Dahoud. 2023. From GPT to AutoGPT: a brief attention in NLP processing using DL. *Preprint* (2023).
- [33] Luciano Floridi and Massimo Chiriatti. 2020. GPT-3: Its nature, scope, limits, and consequences. *Minds and Machines* 30 (2020), 681–694.
- [34] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [35] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. {Cost-Efficient} Large Language Model Serving for Multi-turn Conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 111–126.
- [36] John Gittins, Kevin Glazebrook, and Richard Weber. 2011. *Multi-armed bandit allocation indices*. John Wiley & Sons.
- [37] Diandian Gu, Yihao Zhao, Yinmin Zhong, Yifan Xiong, Zhenhua Han, Peng Cheng, Fan Yang, Gang Huang, Xin Jin, and Xuanzhe Liu. 2023. ElasticFlow: An elastic serverless training platform for distributed deep learning. In *ACM ASPLOS*.

- [38] Muhammad Usman Hadi, Rizwan Qureshi, Abbas Shah, Muhammad Irfan, Anas Zafar, Muhammad Bilal Shaikh, Naveed Akhtar, Jia Wu, Seyedali Mirjalili, et al. 2023. A survey on large language models: Applications, challenges, limitations, and practical usage. *Authorea Preprints* (2023).
- [39] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Yuhang Dong, Yu Wang, et al. 2024. FlashDecoding++: Faster Large Language Model Inference with Asynchronization, Flat GEMM Optimization, and Heuristics. *Proceedings of Machine Learning and Systems* 6 (2024), 148–161.
- [40] Sirui Hong, Xiaowu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. 2023. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352* (2023).
- [41] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).
- [42] Suhas Jayaram Subramanya, Daiyaan Arfeen, Shouxu Lin, Aurick Qiao, Zhihao Jia, and Gregory R Ganger. 2023. Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling. In *ACM SOSp*.
- [43] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *Comput. Surveys* 55, 12 (2023), 1–38.
- [44] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating LLM hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 1827–1843.
- [45] Hanlei Jin, Yang Zhang, Dan Meng, Jun Wang, and Jinghua Tan. 2024. A comprehensive survey on process-oriented automatic text summarization with exploration of llm-based methods. *arXiv preprint arXiv:2403.02901* (2024).
- [46] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. 2023. Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629* (2023).
- [47] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [48] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
- [49] Suyi Li, Hanfeng Lu, Tianyuan Wu, Minchen Yu, Qizhen Weng, Xusheng Chen, Yizhou Shan, Binhang Yuan, and Wei Wang. 2024. CaraServe: CPU-Assisted and Rank-Aware LoRA Serving for Generative LLM Inference. *arXiv preprint arXiv:2401.11240* (2024).
- [50] Bin Lin, Tao Peng, Chen Zhang, Minmin Sun, Lanbo Li, Hanyu Zhao, Wencong Xiao, Qi Xu, Xiafei Qiu, Shen Li, et al. 2024. Infinite-llm: Efficient llm service for long context with distattention and distributed kvcache. *arXiv preprint arXiv:2401.02669* (2024).
- [51] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *USENIX OSDI*.
- [52] Jiachen Liu, Zhiyu Wu, Jae-Won Chung, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. 2024. Andes: Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services. *arXiv preprint arXiv:2404.16283* (2024).
- [53] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Rae Ying Yee Wong, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. 2023. Specinfer: Accelerating generative llm serving with speculative inference and token tree verification. *arXiv preprint arXiv:2305.09781* 1, 2 (2023), 4.
- [54] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. 2020. Heterogeneity-Aware cluster scheduling policies for deep learning workloads. In *USENIX OSDI*.
- [55] Chengyi Nie, Rodrigo Fonseca, and Zhenhua Liu. 2024. Aladdin: Joint Placement and Scaling for SLO-Aware LLM Serving. *arXiv preprint arXiv:2405.06856* (2024).
- [56] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R Ganger, and Eric P Xing. 2021. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*.
- [57] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soricut, Angeliki Lazari-dou, Orhan Firat, Julian Schrittwieser, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530* (2024).
- [58] Linus Schrage. 1968. A proof of the optimality of the shortest remaining processing time discipline. *Operations Research* 16, 3 (1968), 687–690.
- [59] Ziv Scully and Mor Harchol-Balter. 2021. The Gittins policy in the M/G/1 queue. In *2021 19th International Symposium on Modeling and Optimization in Mobile, Ad hoc, and Wireless Networks (WiOpt)*. IEEE, 1–8.
- [60] Philip Sedgwick. 2012. Pearson’s correlation coefficient. *Bmj* 345 (2012).
- [61] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2024. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems* 36 (2024).
- [62] Zhuocheng Shen. 2024. LLM With Tools: A Survey. *arXiv preprint arXiv:2409.18807* (2024).
- [63] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, et al. 2023. S-lora: Serving thousands of concurrent lora adapters. *arXiv preprint arXiv:2311.03285* (2023).
- [64] Ramesh Sitaraman. 2001. The power of two random choices: A survey of techniques and results. (2001).
- [65] Yuki Sonoda, Ryo Kurokawa, Yuta Nakamura, Jun Kanzawa, Mariko Kurokawa, Yuji Ohizumi, Wataru Gono, and Osamu Abe. 2024. Diagnostic performances of GPT-4o, Claude 3 Opus, and Gemini 1.5 Pro in “Diagnosis Please” cases. *Japanese Journal of Radiology* (2024), 1–5.
- [66] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. 2020. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems* 33 (2020), 3008–3021.
- [67] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llmunix: Dynamic Scheduling for Large Language Model Serving. In *USENIX OSDI*.
- [68] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2024. Teola: Towards End-to-End Optimization of LLM-based Applications. *arXiv preprint arXiv:2407.00326* (2024).
- [69] Oguzhan Topsakal and Tahir Cetin Akinci. 2023. Creating large language model applications utilizing langchain: A primer on developing llm apps fast. In *International Conference on Applied Engineering and Natural Sciences*.
- [70] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [71] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast distributed inference serving for large language

- models. *arXiv preprint arXiv:2305.05920* (2023).
- [72] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
- [73] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. ORCA: A Distributed Serving System for Transformer-Based Generative Models. In *USENIX OSDI*.
- [74] Chengliang Zhang, Minchen Yu, Wei Wang, and Feng Yan. 2019. {MArk}: Exploiting cloud services for {Cost-Effective}, {SLO-Aware} machine learning inference serving. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. 1049–1062.
- [75] Pengfei Zheng, Rui Pan, Tarannum Khan, Shivaram Venkataraman, and Aditya Akella. 2023. Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning. In *USENIX NSDI*.
- [76] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *USENIX OSDI*.